

HW 4 – Reinforcement Learning for Vision-Language Models (GRPO)

MMAI 2026 – Multimodal Machine Learning

Peiran Niu

April 2026

Part 1: Reading & Reflection (20 points)

Handout Questions

1. GRPO vs PPO: GRPO removes the need for a learned value function (critic). Explain how GRPO estimates the baseline for advantage computation without a critic, and what tradeoffs this introduces.
2. Reward Design: Discuss the difference between using a learned reward model vs. a rule-based reward function (e.g., checking if an answer matches ground truth). What are the risks of reward hacking in each case?
3. SFT vs. GRPO: In Homework 3, you fine-tuned a VLM using supervised fine-tuning (SFT) with LoRA. Compare the SFT approach with GRPO training: how does the training signal differ, and when would each be preferred?

Answers

Q1 — GRPO vs PPO

From what I understand, GRPO avoids training a separate critic network. Instead, it just uses a group-based baseline. For a given prompt, it generates G different completions and calculates the mean reward for that specific group. The advantage is basically just how much a specific completion deviates from this group mean. This is really nice because it saves a lot of memory since we don't need to keep a value network around.

However, a tradeoff I noticed is that if all G completions happen to get the exact same reward, the advantage becomes zero. When that happens, the model doesn't really get a learning signal from that prompt, which might slow down training.

Q2 — Reward Design

A learned reward model is helpful for subjective things, but it seems really prone to reward hacking. If the policy finds a weird loophole that the reward model likes, it will just exploit it without actually doing the task properly.

On the other hand, rule-based rewards (like checking if the string exactly matches the ground truth) are much more transparent and easy to debug. But they can be pretty brittle. For example, if the model outputs the right answer but messes up the spacing or formatting slightly, it gets a zero. I guess that's why we need partial credit or a separate format check to smooth things out.

Q3 — SFT vs GRPO

SFT gives direct supervision on a token level, so it usually converges a lot faster and is more stable. It seems really good for teaching the model basic formatting and telling it what the task actually is. GRPO is more about trial and error based on a scalar reward, which makes the training a bit noisier.

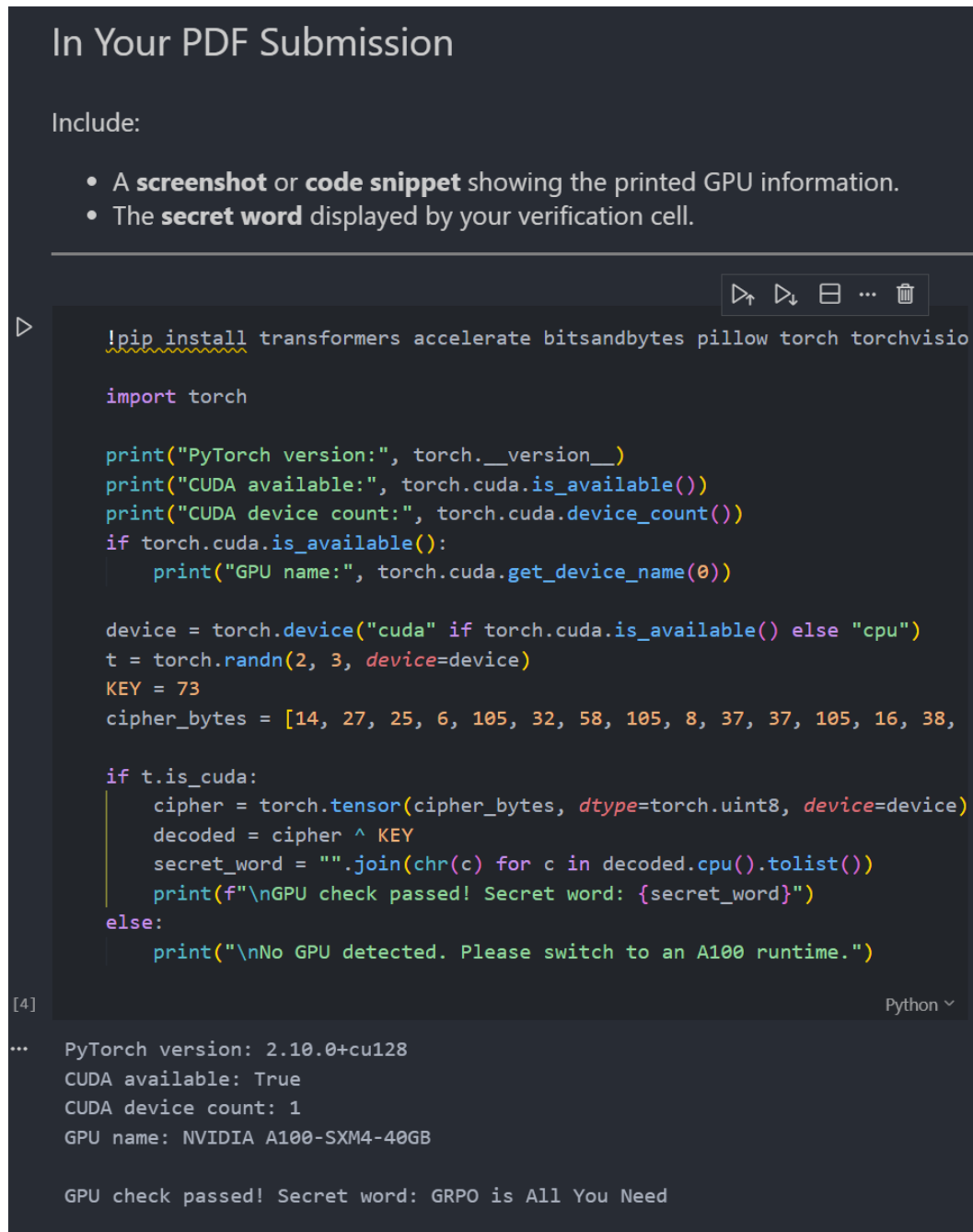
I think a standard approach is to use SFT first to get the output format right, and then use GRPO later to push the model to optimize specific metrics (like exact match accuracy) that are hard to express with just cross-entropy loss.

Part 2: Implementing and Training GRPO (100 points)

Problem 1: GPU Verification

I ran the environment in Google Colab with an A100 GPU. After installing the packages, the check passed and printed the secret word.

Secret word: GRPO is All You Need



The screenshot shows a Google Colab notebook interface. At the top, there's a header "In Your PDF Submission" with a list of requirements: a screenshot or code snippet showing GPU information, and the secret word displayed by the verification cell. Below this is a code cell with Python code for GPU verification. The code installs necessary packages, imports torch, checks PyTorch version and CUDA availability, and prints GPU details. It then generates a cipher using a key and a list of bytes, decodes it on the GPU, and prints the secret word. The output cell shows the execution results: PyTorch version, CUDA availability, GPU device count, GPU name, and the decoded secret word.

```
!pip install transformers accelerate bitsandbytes pillow torch torchvision

import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("CUDA device count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
t = torch.randn(2, 3, device=device)
KEY = 73
cipher_bytes = [14, 27, 25, 6, 105, 32, 58, 105, 8, 37, 37, 105, 16, 38,

if t.is_cuda:
    cipher = torch.tensor(cipher_bytes, dtype=torch.uint8, device=device)
    decoded = cipher ^ KEY
    secret_word = "".join(chr(c) for c in decoded.cpu().tolist())
    print(f"\nGPU check passed! Secret word: {secret_word}")
else:
    print("\nNo GPU detected. Please switch to an A100 runtime.")
```

[4] Python ▾

```
... PyTorch version: 2.10.0+cu128
CUDA available: True
CUDA device count: 1
GPU name: NVIDIA A100-SXM4-40GB

GPU check passed! Secret word: GRPO is All You Need
```

Figure 2.1: GPU verification output.

Problem 2: Dataset Preparation

I reused the MILP heatmap dataset from HW3. The training split was used for GRPO, and I kept the test split for the evaluation at the end.

Problem 3: Understanding GRPO

Handout Questions

1. In your own words, explain why the group-based advantage normalization works as a baseline. What happens when all G completions for a prompt receive the same reward?
2. What is the role of the clipping term $\text{clip}(\rho, 1-\epsilon, 1+\epsilon)$? What could go wrong during training without it?

Answers

Q1: The group mean acts as a simple baseline. By comparing a completion's reward to the average of its group, the model knows if it did better or worse than its other attempts for that specific prompt. If all G completions get the exact same reward, the advantage is zero, meaning the gradient is zero and it doesn't really learn anything from that step.

Q2: The clipping term basically stops the model from making updates that are too drastic in a single step. Without it, a few really good or really bad tokens could cause huge shifts in the policy, making the training unstable or causing it to collapse.

Connection to Implementation (Problem 4)

In the `compute_grpo_advantage` function, I just followed this logic by grouping samples by `group_ids`, getting the mean and standard deviation, and then normalizing. I set `std=1.0` for singleton groups just to avoid division by zero errors.

Problem 4: GRPO Advantage Computation

I implemented `compute_grpo_advantage` as described in the handout. The details are mostly covered in the Problem 3 answers.

Problem 5: Reward Functions

I used two reward functions for this run. The accuracy reward checks the MILP fields. Since a strict exact match is too hard at the beginning, I gave partial credit. If it got 4 out of 6 fields right, it got a 0.667 reward. The format reward is simpler—it just gives a 1.0 if the output parses correctly as a MILP line, otherwise 0.

Problem 6: Training Dataset for GRPO

The `build_grpo_dataset` function just maps the images and questions into a HuggingFace Dataset. I appended an instruction suffix asking the model to reason step-by-step before giving the answer line.

Problem 7: GRPO Training

Handout Questions

1. Report the hyperparameter settings you used to obtain your best model. How did reward-related metrics change over training?
2. Compare the training dynamics of GRPO with SFT/LoRA. Which appeared to converge faster, and which produced better results?
3. Ablation — no-SFT vs SFT-warmup GRPO: Compare held-out accuracy, format, reward curves, and qualitative outputs.

Training Setup

I used Qwen3-VL-2B-Instruct with LoRA. The main hyperparameters were: NUM_GENERATIONS = 4, MAX_STEPS = 50, LEARNING_RATE = 1e-5, and EPSILON = 0.2. I basically ran two setups to compare: one cold-start GRPO directly on the base model, and one where I loaded the SFT adapter first (SFT-warmup).

To keep things organized, I used switches in the notebook to separate the cold-start run and the SFT-warmup run, saving their logs in different folders.

Run A — Cold Start (no_sft)

For the pure GRPO run without SFT, the surrogate loss just seemed to bounce around zero the whole time. I think this is normal since advantages are normalized per group, so loss isn't supposed to just go down smoothly. It struggled quite a bit with getting the exact format right initially, but it did try to write out step-by-step reasoning.

Logged metrics (from grpo_log_history_no_sft.json): 50 steps (step 1–50). Surrogate loss ranged from -0.2549 to 0.6362 with a mean of 0.0420; loss was near zero on 6 of 50 steps. Accuracy reward (mean): started at 0.4583, ended at 0.0000, with an overall average of 0.2179 (range 0.0000–0.4583). Format reward (mean): started at 1.0000, ended at 0.0000.

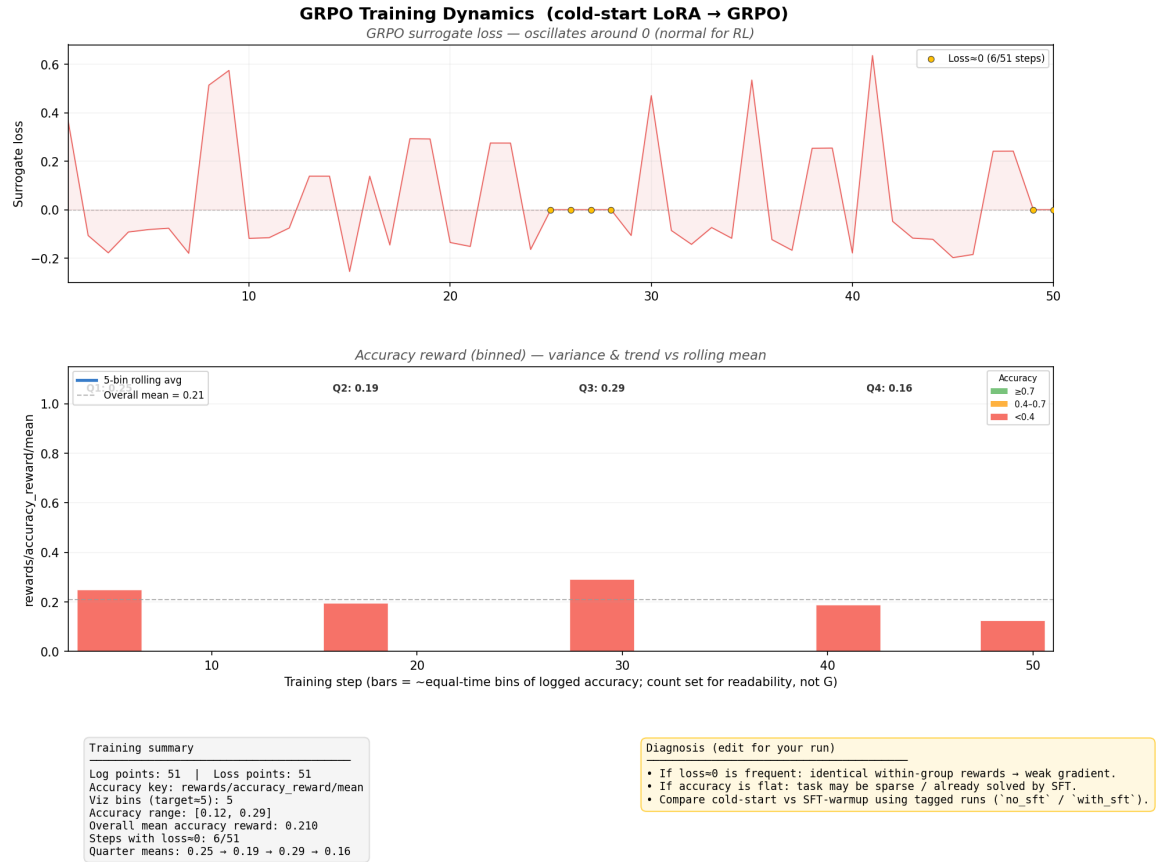


Figure 7.1: GRPO training analysis — cold start (no SFT).

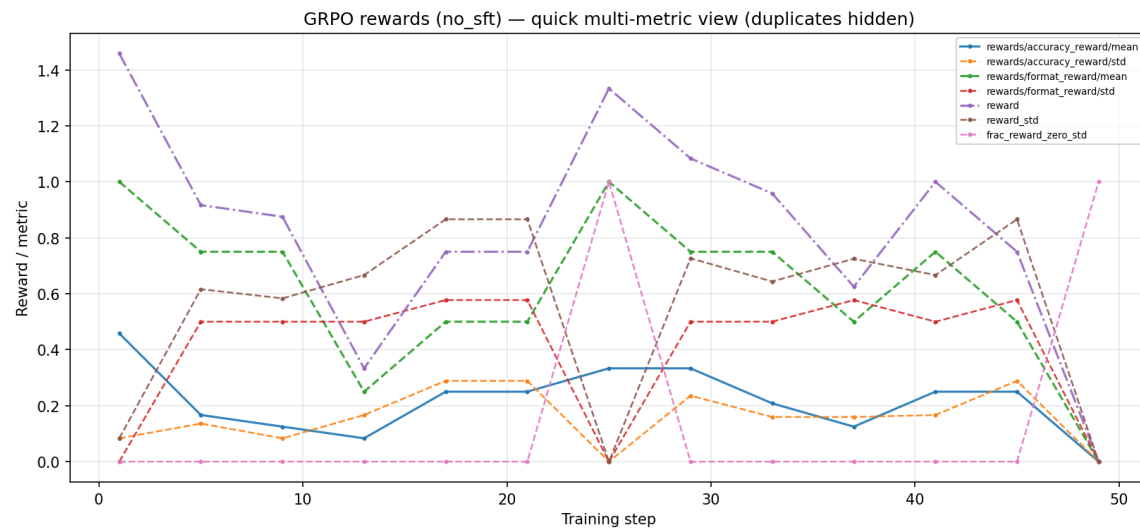


Figure 7.2: Multi-metric reward curves — cold start.

Run B — SFT Warmup then GRPO (with_sft)

Starting from the SFT adapter made a huge difference. The format reward was pretty much perfect right from the start because SFT already taught it the structure. The outputs from this

run were generally much shorter—it looks like SFT taught it to just output the answer directly without much reasoning.

Logged metrics (from `grpо_log_history_with_sft.json`): 50 steps (step 1–50). Surrogate loss ranged from -0.3917 to 0.3782 with a mean of 0.0041; loss was near zero on 4 of 50 steps. Accuracy reward (mean): started at 0.6250, ended at 0.7083, with an overall average of 0.4936 (range 0.2917–0.7083). Format reward (mean): started at 1.0000, ended at 1.0000.

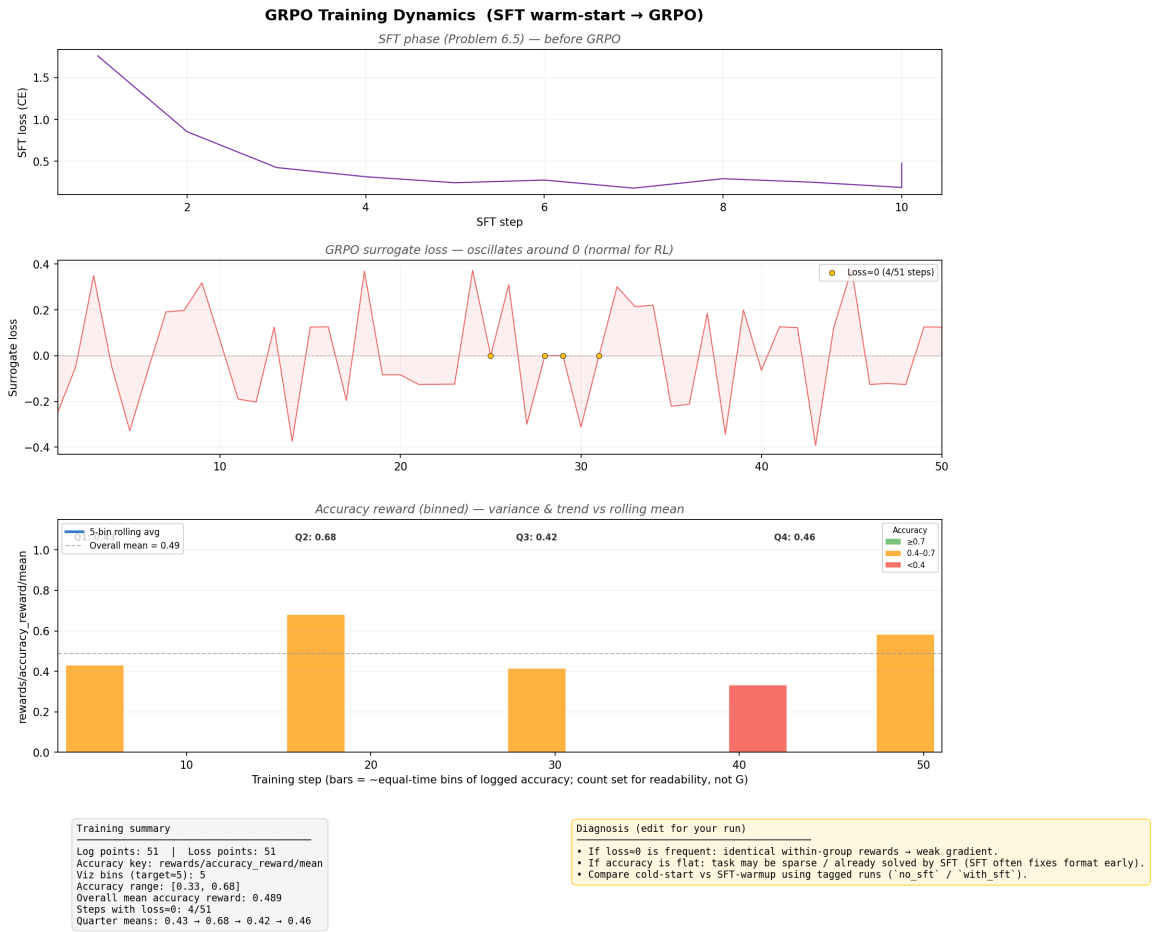


Figure 7.3: GRPO training analysis — SFT warmup then GRPO.

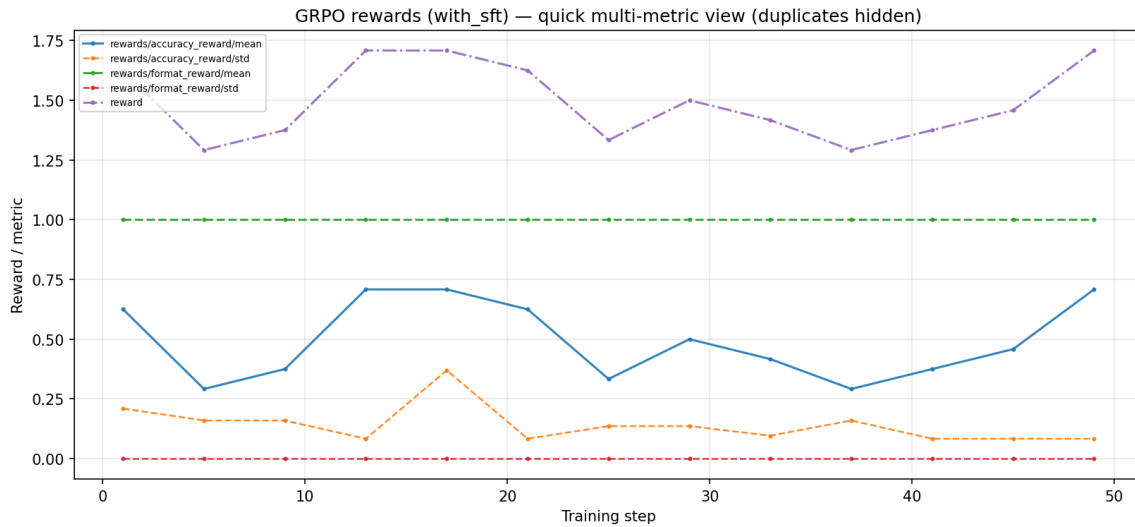


Figure 7.4: Multi-metric reward curves — SFT warmup.

Ablation Comparison (Question 3)

Both runs used identical GRPO hyperparameters and differed only in whether SFT was applied beforehand. The cold-start run averaged an accuracy reward of 0.2179 over training (starting at 0.4583, ending at 0.0000), while the SFT-warmup run averaged 0.4936 (starting at 0.6250, ending at 0.7083). Format reward was already near 1.0 for the SFT-warmup run from the start. These numbers should be read together with the reward curves in Figures 7.1–7.4.

Both runs used the exact same GRPO hyperparameters, so the only difference is whether it saw the SFT data first.

GRPO vs SFT Training Dynamics (Question 2)

As expected, SFT converges really fast because it has direct labels. GRPO takes longer and looks a lot noisier. The SFT-warmup run definitely gave better quantitative results overall. But one weird thing I noticed is that higher accuracy rewards didn't mean longer reasoning. The cold-start model was actually more verbose, while the SFT-warmup model just spit out concise answers.

Problem 8: Post-Training Evaluation

To evaluate, I loaded the LoRA adapters on the held-out test split using a temperature of 0.7.

Problem 8.2 — Held-Out Results: Cold-Start GRPO

Run A — Cold-start GRPO. Overall, it didn't do great on exact match. The format compliance was okay (around 75%), but the mean partial score was pretty low. It struggled a lot with counting variables and constraints, though it did slightly better on categorical things like density.

Problem 8.2 — Held-Out Results: SFT Warmup GRPO

Run B — SFT warmup then GRPO. This configuration did much better. Format compliance was 100%, and the mean partial score went up to 0.667. For example, in one test sample, it got 5 out

of 6 fields right, only missing the objective direction. The answers were very clean and to the point.

Problem 8.2 — Ablation Discussion

Comparing the two runs, the SFT-warmup one is obviously better if we just look at the scores. But the cold-start model was kind of interesting because it produced more explicit reasoning steps. It feels like SFT forced the model to be brief, and GRPO didn't really reverse that, whereas the cold-start model had to explore more on its own.

Problem 8.2 — Baselines: Zero-Shot and SFT-Only

Base (zero-shot, no adapters): n=8 exact=0.0% format=87.5% mean_partial=0.381

SFT-only (sft-output/): n=8 exact=0.0% format=100.0% mean_partial=0.646

Problem 8.3 — Side-by-Side Generation Excerpts

Here are the outputs I got for Sample 1 for both GRPO runs to show the difference:

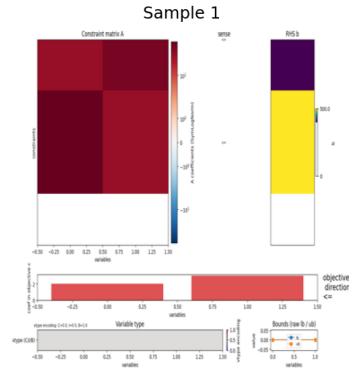
Variant	Excerpt (truncated)
GRPO cold-start	Reasoning: 1. Let's count the variables... Answer: Variables: 4, Constraints: 5, Density: high, Binary: 0%, Integer: 0%, Objective: minimize
GRPO with SFT warmup	Variables: 2, Constraints: 2, Density: high, Binary: 0%, Integer: 100%, Objective: maximize

It seems like the cold-start model tries to write out its thought process, maybe because the RL environment rewards it when it eventually guesses right after exploring. The SFT-warmup model is much more direct. It got the format perfectly but missed the objective direction here, which gave it a 5/6 partial score.

Problem 8.3 — Reflection

One big takeaway for me was that looking at the surrogate loss isn't very helpful for GRPO—it just oscillates around zero most of the time. The reward curves are what actually tell you if it's learning.

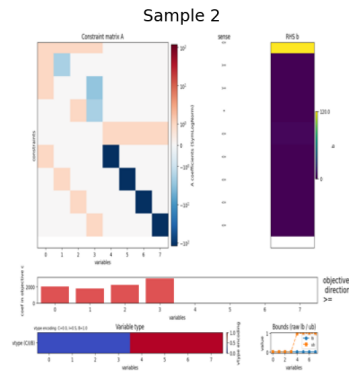
Also, initializing with SFT makes training much easier and leads to better final scores, but it seems to stop the model from generating long 'chain-of-thought' reasoning. It was pretty cool to see how pure RL (the cold-start run) leads to more verbose, exploratory behavior, even if it doesn't always get the right answer in the end. I think for a real application, SFT followed by GRPO is definitely the way to go to get reliable formatting and accuracy.



Question:
Describe the structure of this MILP heatmap: number of variables, number of constraints, matrix density, fraction of binary variables, fraction of integer variables, and objective direction (minimize or maximize).

Model output:
Variables: 2, Constraints: 2, Density: high, Binary: 0%, Integer: 100%, Objective: maximize

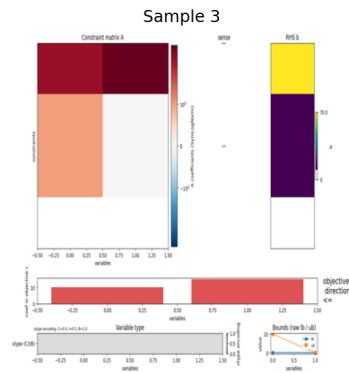
Extracted: 'Variables: 2, Constraints: 2, Density: high, Binary: 0%, Integer: 100%, Objective: maximize'
Ground truth: 'Variables: 2, Constraints: 2, Density: high, Binary: 0%, Integer: 100%, Objective: minimize'
Exact: False | Format: True | Partial: 0.833



Question:
Describe the structure of this MILP heatmap: number of variables, number of constraints, matrix density, fraction of binary variables, fraction of integer variables, and objective direction (minimize or maximize).

Model output:
Variables: 8, Constraints: 8, Density: high, Binary: 0%, Integer: 100%, Objective: minimize

Extracted: 'Variables: 8, Constraints: 8, Density: high, Binary: 0%, Integer: 100%, Objective: minimize'
Ground truth: 'Variables: 8, Constraints: 9, Density: medium, Binary: 50%, Integer: 0%, Objective: maximize'
Exact: False | Format: True | Partial: 0.167



Question:
Describe the structure of this MILP heatmap: number of variables, number of constraints, matrix density, fraction of binary variables, fraction of integer variables, and objective direction (minimize or maximize).

Model output:
Variables: 2, Constraints: 2, Density: high, Binary: 0%, Integer: 100%, Objective: maximize

Extracted: 'Variables: 2, Constraints: 2, Density: high, Binary: 0%, Integer: 100%, Objective: maximize'
Ground truth: 'Variables: 2, Constraints: 2, Density: high, Binary: 0%, Integer: 100%, Objective: minimize'
Exact: False | Format: True | Partial: 0.833

Figure 8.1: Example test outputs.